

INFORME FINAL

BASES DE DATOS

Proyecto: sienashopping — Base de Datos Relacional

Integrantes

Dolly Juliana Aguirre

Julian Carranza Ruiz

Fredy Estiven Pinilla

Hugo Fabián Mora

Tutora: Elsa Junca

Ficha de Formación: 3177445

Programa: ADSO — Análisis y Desarrollo de Software

Centro de Diseño y Metrología

Bogotá D.C.

Abril 2026

Tabla de Contenido

1. Introducción
2. Normalización de la Base de Datos
3. Diagrama ER
4. Modelo MER
5. Script de la Base de Datos
 - 5.1. Creación de Tablas e Inserción de Datos
 - 5.2. Consultas JOIN
6. Conclusiones

1. Introducción

El presente documento reúne el desarrollo completo del diseño y construcción de una base de datos relacional para una tienda de ropa. En él se describen las etapas fundamentales del proceso, desde el análisis y organización de la información hasta la implementación mediante sentencias SQL.

El objetivo principal es demostrar la correcta aplicación de conceptos como normalización, modelado entidad-relación y estructuración de tablas, garantizando una base de datos organizada, coherente y libre de redundancias. Asimismo, se presentan los diagramas y el script final que permiten visualizar y ejecutar la estructura diseñada.

La base de datos diseñada corresponde a una tienda de ropa y contempla entidades como productos, categorías, usuarios, facturas, carrito de compras, promociones, notificaciones y reseñas. Cada una de estas entidades fue analizada y normalizada siguiendo las reglas de las formas normales, garantizando integridad referencial y eficiencia en el almacenamiento de datos.

Este trabajo integra tanto la parte teórica como la práctica, evidenciando el proceso completo de creación de una base de datos funcional, lista para ser implementada en un entorno de desarrollo de software.

2. Normalización de la Base de Datos

La normalización es un proceso sistemático que consiste en organizar los atributos y relaciones de una base de datos con el fin de minimizar la redundancia de datos y evitar anomalías de inserción, actualización y eliminación. A continuación se presenta el proceso de normalización aplicado al proyecto de la tienda de ropa.

Primera Forma Normal (1FN)

Para que una tabla se encuentre en 1FN debe cumplir: todos los atributos deben ser atómicos (indivisibles), no deben existir grupos repetitivos, y cada fila debe ser única. Se identificaron las siguientes entidades iniciales:

- CLIENTES / USUARIOS
- CIUDAD
- PRODUCTO
- CATEGORÍA
- PROMOCIÓN

Se asignaron claves primarias: ID_CLIENTES, ID_USUARIO, ID_PRODUCTO, ID_CATEGORIA. Se eliminaron atributos multivaluados asegurando que cada celda contuviera un único valor.

Segunda Forma Normal (2FN)

Para la 2FN, además de cumplir 1FN, todos los atributos no clave deben depender completamente de la clave primaria (eliminación de dependencias parciales). Se identificaron y separaron las siguientes tablas:

- CLIENTES / USUARIOS — separados por responsabilidad
- PRODUCTO — con referencia a CATEGORÍA
- FACTURA — con referencia a VENTA e ID_FACTURA
- VENTA — con clave ID_VENTA relacionada a ID_FACTURA

Tercera Forma Normal (3FN)

Para la 3FN, además de cumplir 2FN, no deben existir dependencias transitivas (un atributo no clave no debe depender de otro atributo no clave). Se consolidó el modelo final con las siguientes tablas:

Tabla	Clave Primaria	Relaciones	Atributos Principales
T_USUARIO	ID_USUARIO	T_CIUDAD	nombre, apellido, correo, password, teléfono, dirección
T_CIUDAD	ID_CIUDAD	T_USUARIO	ciudad, localidad, barrio, dirección
T_CATEGORIA	ID_CATEGORIA	T_PRODUCTO	categoría
T_PROMOCION	ID_PROMOCION	T_PRODUCTO, T_NOTIFICACION	título, descripción, descuento, fechas, estado
T_PRODUCTO	ID_PRODUCTO	T_CATEGORIA, T_PROMOCION	nombre, descripción, color, talla, precio_costo, precio_venta
T_FACTURA	ID_FACTURA	T_USUARIO, T_PRODUCTO	fecha, método_pago, cantidad, valor_unitario, ciudad, dirección
T_CARRITO	ID_CARRITO	T_USUARIO, T_PRODUCTO	cantidad, fecha_agregado
T_NOTIFICACION	ID_NOTIFICACION	T_USUARIO, T_PROMOCION	fecha_envio
T_RESENA	ID_RESENA	T_USUARIO, T_PRODUCTO	calificación, comentario, fecha

La normalización hasta la 3FN garantiza que la base de datos esté libre de redundancias, anomalías y dependencias transitivas, logrando un diseño eficiente y escalable.

3. Diagrama ER

El Diagrama Entidad-Relación (ER) representa gráficamente las entidades del sistema, sus atributos y las relaciones existentes entre ellas. Este diagrama constituye la base conceptual del diseño de la base de datos de la tienda de ropa.

Las principales entidades identificadas son: T_USUARIO, T_CIUDAD, T_PRODUCTO, T_CATEGORIA, T_PROMOCION, T_FACTURA, T_CARRITO, T_NOTIFICACION y T_RESEÑA. Cada entidad se relaciona con otras a través de llaves foráneas que garantizan la integridad referencial del sistema.

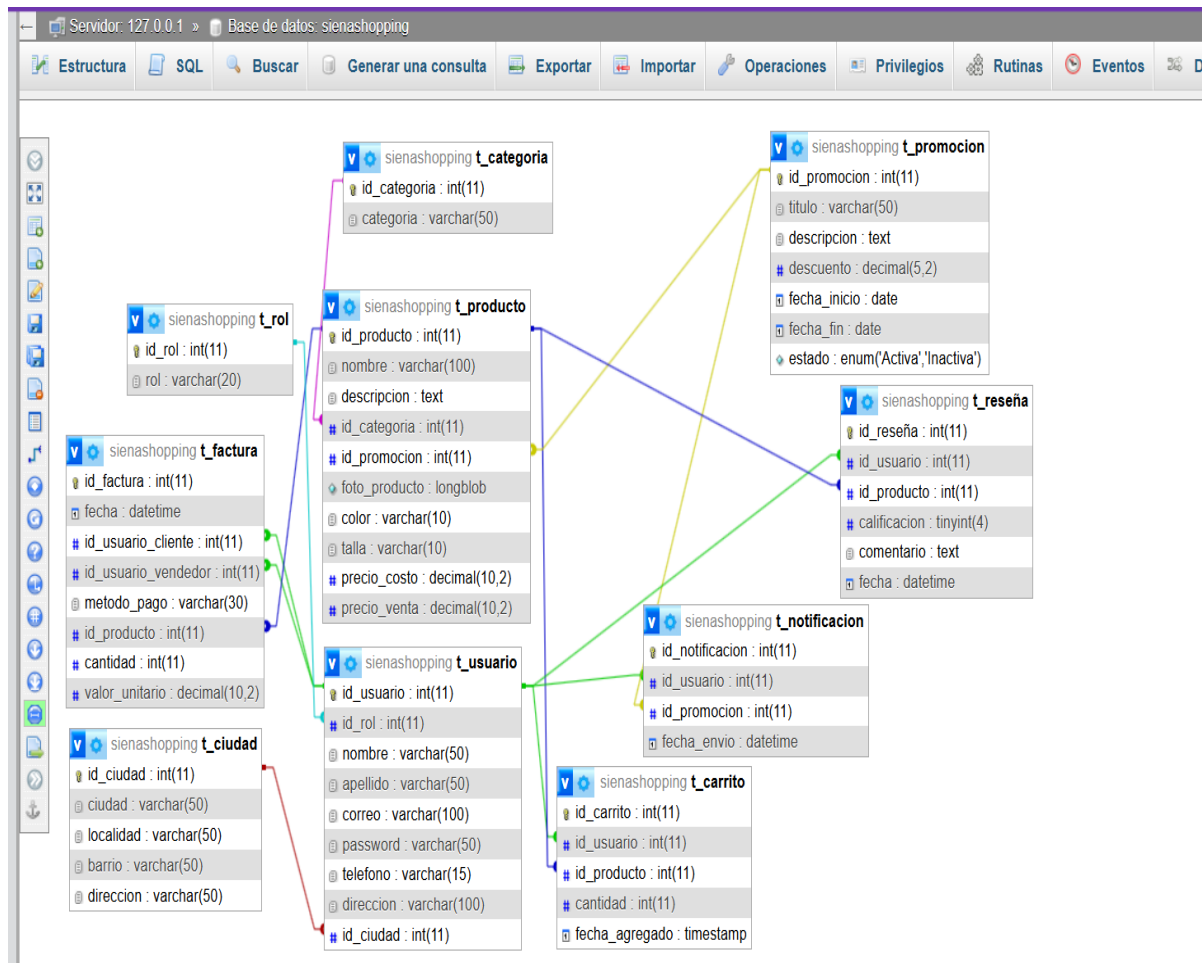


Figura 1. Diagrama Entidad-Relación (ER) — Base de Datos Tienda de Ropa

4. Modelo MER

El Modelo Entidad-Relación (MER) es la representación lógica que define con precisión las entidades, sus atributos, claves primarias, claves foráneas y las cardinalidades de las relaciones. Este modelo sirve como guía directa para la implementación física de la base de datos.

A continuación se presentan las relaciones de cardinalidad identificadas en el modelo:

- Un USUARIO pertenece a una CIUDAD (N:1).
- Un PRODUCTO pertenece a una CATEGORÍA (N:1) y puede tener una PROMOCIÓN (N:1).
- Una FACTURA referencia a un CLIENTE, un VENDEDOR (ambos USUARIO) y un PRODUCTO (N:1).
- Un CARRITO pertenece a un USUARIO y referencia un PRODUCTO (N:1).
- Una NOTIFICACION relaciona a un USUARIO con una PROMOCIÓN (N:1).
- Una RESEÑA es realizada por un USUARIO sobre un PRODUCTO (N:1).

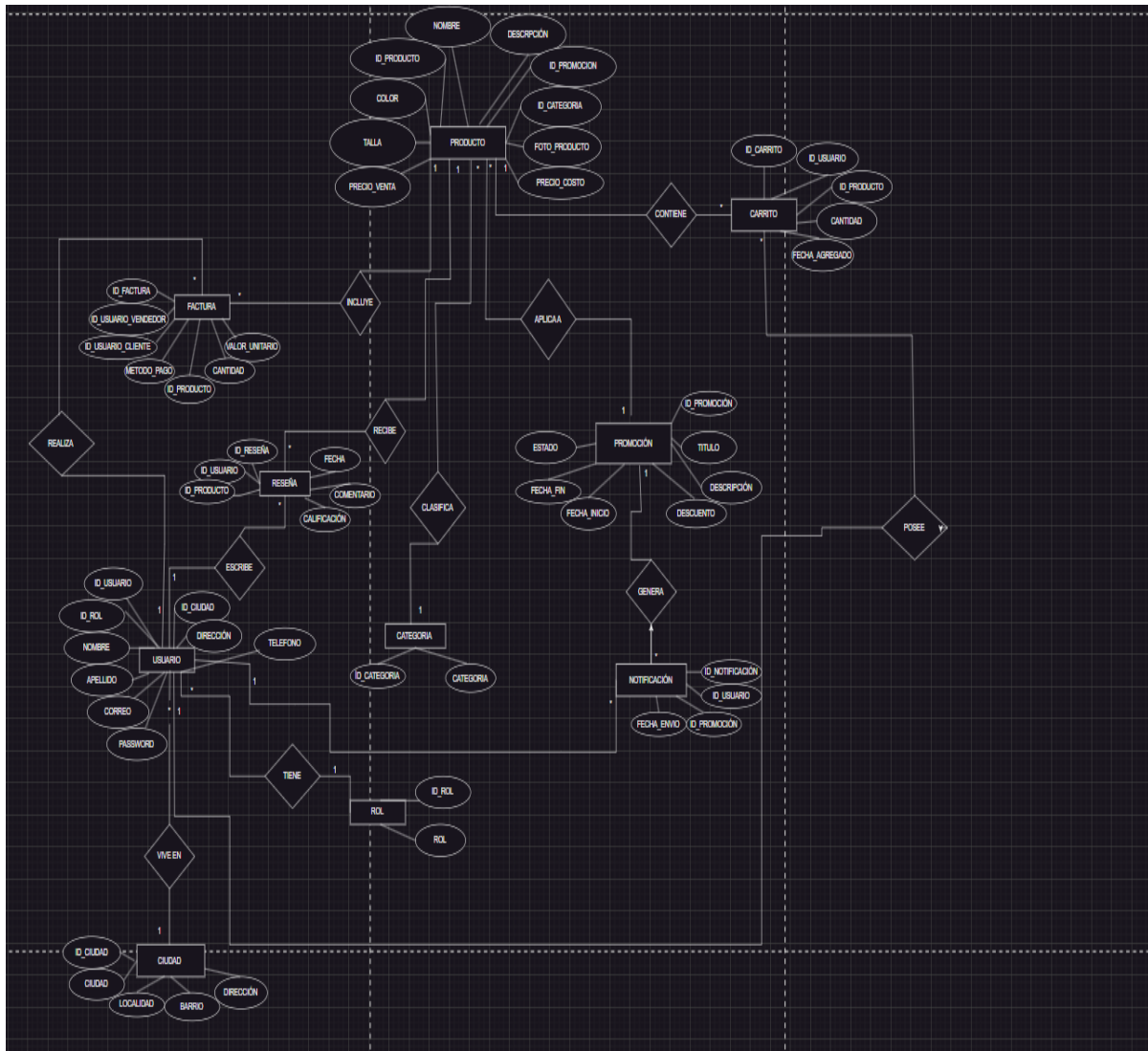


Figura 2. Modelo Entidad-Relación (MER) — Base de Datos Tienda de Ropa

5. Script de la Base de Datos

En esta sección se presenta el script completo de la base de datos relacional denominada 'tienda'. El script incluye la creación de todas las tablas con sus restricciones e integridad referencial, la inserción de datos de prueba, y las consultas SQL con diferentes tipos de JOIN para extraer información útil del sistema.

5.1. Creación de Tablas e Inserción de Datos

A continuación se presenta el script SQL completo con la definición de las 9 tablas de la base de datos y los registros de prueba insertados en cada una de ellas:

```
SET FOREIGN_KEY_CHECKS = 0;

-- Tabla: t_categoria
CREATE TABLE `t_categoria` (
  `id_categoria` int(11) NOT NULL AUTO_INCREMENT,
  `categoria` varchar(50) NOT NULL,
  PRIMARY KEY (`id_categoria`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO `t_categoria` VALUES
(1,'Camisetas'),(2,'Pantalones'),(3,'Vestidos'),(4,'Chaquetas'),(5,'Accesorios');

-- Tabla: t_ciudad
CREATE TABLE `t_ciudad` (
  `id_ciudad` int(11) NOT NULL AUTO_INCREMENT,
  `ciudad` varchar(50) NOT NULL,
  `localidad` varchar(50) NOT NULL,
  `barrio` varchar(50) NOT NULL,
  `direccion` varchar(50) NOT NULL,
  PRIMARY KEY (`id_ciudad`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO `t_ciudad` VALUES
(1,'Bogotá','Usaquén','Santa Bárbara','Calle 119 # 15-30'),
```

```
(2,'Bogotá','Suba','Tibabuyes','Carrera 170 # 12-30'),  
(4,'Bogotá','Fontibón','Centro Fontibón','Carrera 99 # 23-45'),  
(5,'Bogotá','Fontibón','Versalles','Calle 22 # 103-15'),  
(6,'Bogotá','Suba','Lisboa','Carrera 150D # 143-21');
```

-- Tabla: t_promocion

```
CREATE TABLE `t_promocion` (  
  `id_promocion` int(11) NOT NULL AUTO_INCREMENT,  
  `titulo` varchar(50) NOT NULL,  
  `descripcion` text NOT NULL,  
  `descuento` decimal(5,2) NOT NULL,  
  `fecha_inicio` date NOT NULL,  
  `fecha_fin` date NOT NULL,  
  `estado` enum('Activa','Inactiva') NOT NULL DEFAULT 'Activa',  
  PRIMARY KEY (`id_promocion`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

INSERT INTO `t\_promocion` VALUES

```
(1,'Temporada Verano','Descuento en ropa de verano',15.00,'2026-03-01','2026-03-31','Activa'),  
(2,'Liquidación Total','Liquidación de inventario',30.00,'2026-04-01','2026-04-15','Inactiva'),  
(3,'Semana Fashion','Descuentos en vestidos y faldas',20.00,'2026-05-01','2026-05-07','Activa'),  
(4,'Black Friday','Mega descuento en toda la tienda',50.00,'2026-11-28','2026-11-29','Inactiva'),  
(5,'Inicio de Año','Rebajas en chaquetas',10.00,'2026-01-02','2026-01-15','Inactiva');
```

-- Tabla: t_usuario

```
CREATE TABLE `t_usuario` (  
  `id_usuario` int(11) NOT NULL AUTO_INCREMENT,  
  `id_rol` int(11) NOT NULL,  
  `nombre` varchar(50) NOT NULL,  
  `apellido` varchar(50) NOT NULL,  
  `correo` varchar(100) NOT NULL,  
  `password` varchar(50) NOT NULL,  
  `telefono` varchar(15) NOT NULL,  
  `direccion` varchar(100) NOT NULL,
```

```

`id_ciudad` int(11) NOT NULL,
PRIMARY KEY (`id_usuario`),
UNIQUE KEY `correo` (`correo`),
CONSTRAINT `fk_usuario_ciudad` FOREIGN KEY (`id_ciudad`) REFERENCES
`t_ciudad` (`id_ciudad`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `t_usuario` VALUES
(1,1,'Carlos','Ramírez','carlos.ramirez@email.com','pass1234','3101234567','Calle 10 # 5-20',1),
(2,2,'María','López','maria.lopez@email.com','pass5678','3209876543','Carrera 8 # 12-45',1),
(3,2,'Andrés','Torres','andres.torres@email.com','pass9012','3153456789','Av. 6 # 30-10',1),
(4,1,'Valentina','Gómez','valentina.gomez@email.com','pass3456','3007891234','Calle 72 # 20-05',1),
(5,2,'Luis','Herrera','luis.herrera@email.com','pass7890','3116547890','Carrera 15 # 8-60',1);

```

-- Tabla: t_producto

```

CREATE TABLE `t_producto` (
`id_producto` int(11) NOT NULL AUTO_INCREMENT,
`nombre` varchar(100) NOT NULL,
`descripcion` text NOT NULL,
`id_categoria` int(11) NOT NULL,
`id_promocion` int(11) DEFAULT NULL,
`foto_producto` longblob DEFAULT NULL,
`color` varchar(10) NOT NULL,
`talla` varchar(10) NOT NULL,
`precio_costo` decimal(10,2) NOT NULL,
`precio_venta` decimal(10,2) NOT NULL,
PRIMARY KEY (`id_producto`),
CONSTRAINT `fk_producto_categoria` FOREIGN KEY (`id_categoria`) REFERENCES
`t_categoria` (`id_categoria`),
CONSTRAINT `fk_producto_promocion` FOREIGN KEY (`id_promocion`)
REFERENCES `t_promocion` (`id_promocion`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `t_producto`
(`id_producto`,`nombre`,`descripcion`,`id_categoria`,`id_promocion`,`color`,`talla`,`precio_
_costo`,`precio_venta`) VALUES
(1,'Camiseta Básica Blanca','Camiseta de algodón 100%, cuello
redondo',1,1,'Blanco','M',25000.00,55000.00),
(2,'Jean Slim Fit Azul','Pantalón jean corte slim, tela strech
cómoda',2,NULL,'Azul','32',60000.00,120000.00),
(3,'Vestido Floral Rojo','Vestido midi estampado flores, manga
corta',3,3,'Rojo','S',45000.00,95000.00),
(4,'Chaqueta Cuero Negra','Chaqueta sintética estilo biker, cierre
metálico',4,NULL,'Negro','L',90000.00,180000.00),
(5,'Cinturón Tejido Café','Cinturón artesanal tejido a mano, hebilla
dorada',5,5,'Café','L',15000.00,35000.00);

```

-- Tabla: t_carrito

```

CREATE TABLE `t_carrito` (
  `id_carrito` int(11) NOT NULL AUTO_INCREMENT,
  `id_usuario` int(11) NOT NULL,
  `id_producto` int(11) NOT NULL,
  `cantidad` int(11) NOT NULL DEFAULT 1,
  `fecha_agregado` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id_carrito`),
  CONSTRAINT `fk_carrito_usuario` FOREIGN KEY (`id_usuario`) REFERENCES
`t_usuario`(`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `fk_carrito_producto` FOREIGN KEY (`id_producto`) REFERENCES
`t_producto`(`id_producto`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `t_carrito` VALUES
(1,1,1,2,'2026-03-10 14:15:00'),(2,2,3,1,'2026-03-10 16:30:00'),
(3,3,2,1,'2026-03-11 19:00:00'),(4,4,5,3,'2026-03-12 21:45:00'),
(5,5,4,1,'2026-03-13 15:20:00');

```

-- Tabla: t_factura

```

CREATE TABLE `t_factura` (
  `id_factura` int(11) NOT NULL AUTO_INCREMENT,
  `fecha` datetime NOT NULL DEFAULT current_timestamp(),
  `id_usuario_cliente` int(11) NOT NULL,
  `id_usuario_vendedor` int(11) NOT NULL,

```

```

`metodo_pago` varchar(30) NOT NULL,
`id_producto` int(11) NOT NULL,
`cantidad` int(11) NOT NULL DEFAULT 1,
`valor_unitario` decimal(10,2) NOT NULL,
`ciudad` varchar(50) NOT NULL DEFAULT 'Bogotá',
`direccion` varchar(100) NOT NULL DEFAULT '',
PRIMARY KEY (`id_factura`),
CONSTRAINT `fk_factura_cliente` FOREIGN KEY (`id_usuario_cliente`)
REFERENCES `t_usuario`(`id_usuario`),
CONSTRAINT `fk_factura_vendedor` FOREIGN KEY (`id_usuario_vendedor`)
REFERENCES `t_usuario`(`id_usuario`),
CONSTRAINT `fk_factura_producto` FOREIGN KEY (`id_producto`) REFERENCES
`t_producto`(`id_producto`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `t_factura` VALUES
(1,'2026-03-10 10:00:00',1,2,'Tarjeta de Crédito',1,2,55000.00,'Bogotá','Calle 90 # 11-13'),
(2,'2026-03-11 13:00:00',3,2,'Transferencia',3,1,95000.00,'Bogotá','Carrera 170 # 12-30'),
(3,'2026-03-11 15:30:00',4,5,'Efectivo',5,3,35000.00,'Bogotá','Carrera 99 # 23-45'),
(4,'2026-03-12 11:00:00',1,5,'PSE',2,1,120000.00,'Bogotá','Calle 22 # 103-15'),
(5,'2026-03-13 16:00:00',3,2,'Tarjeta Débito',4,1,180000.00,'Bogotá','Carrera 150D # 143-21');

```

-- Tabla: t_notificacion

```

CREATE TABLE `t_notificacion` (
`id_notificacion` int(11) NOT NULL AUTO_INCREMENT,
`id_usuario` int(11) NOT NULL,
`id_promocion` int(11) NOT NULL,
`fecha_envio` datetime NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (`id_notificacion`),
CONSTRAINT `fk_notificacion_usuario` FOREIGN KEY (`id_usuario`) REFERENCES
`t_usuario`(`id_usuario`) ON DELETE CASCADE,
CONSTRAINT `fk_notificacion_promocion` FOREIGN KEY (`id_promocion`)
REFERENCES `t_promocion`(`id_promocion`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

INSERT INTO `t_notificacion` VALUES
(1,1,1,'2026-03-01 08:00:00'),(2,2,3,'2026-05-01 09:00:00'),

```

```

(3,3,1,'2026-03-01 08:05:00'),(4,4,5,'2026-01-02 07:30:00'),
(5,5,3,'2026-05-01 09:10:00');

-- Tabla: t_resena
CREATE TABLE `t_resena` (
  `id_resena` int(11) NOT NULL AUTO_INCREMENT,
  `id_usuario` int(11) NOT NULL,
  `id_producto` int(11) NOT NULL,
  `calificacion` tinyint(4) NOT NULL CHECK (`calificacion` BETWEEN 1 AND 5),
  `comentario` text NOT NULL,
  `fecha` datetime NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id_resena`),
  CONSTRAINT `fk_resena_usuario` FOREIGN KEY (`id_usuario`) REFERENCES
`t_usuario`(`id_usuario`) ON DELETE CASCADE,
  CONSTRAINT `fk_resena_producto` FOREIGN KEY (`id_producto`) REFERENCES
`t_producto`(`id_producto`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO `t_resena` VALUES
(1,1,1,5,'Excelente calidad, muy suave al tacto. La recomiendo.','2026-03-11 08:00:00'),
(2,2,3,4,'Muy bonito el vestido, el color es igual al de la foto.','2026-03-11 10:30:00'),
(3,3,2,3,'El jean está bien pero la talla corre un poco grande.','2026-03-12 12:15:00'),
(4,4,5,5,'El cinturón es hermoso, ideal para regalar. Llegó perfecto.','2026-03-13
17:00:00'),
(5,5,4,4,'La chaqueta es de buena calidad, el envío fue rápido.','2026-03-14 09:45:00');

SET FOREIGN_KEY_CHECKS = 1;
COMMIT;

```

5.2. Consultas JOIN

Las siguientes consultas SQL demuestran el uso de diferentes tipos de JOIN para cruzar información entre las tablas de la base de datos. Se incluyen INNER JOIN simples, LEFT JOIN, JOIN múltiples y JOIN combinados con GROUP BY para consultas analíticas:

```
-- =====
```

-- CONSULTAS JOIN - Base de datos: sienahopping

-- =====

-- 1. INNER JOIN: Productos con su categoría

```
SELECT p.id_producto, p.nombre AS producto, p.color, p.talla, p.precio_venta, c.categoria
FROM t_producto p
INNER JOIN t_categoria c ON p.id_categoria = c.id_categoria;
```

-- 2. INNER JOIN: Productos con su promoción activa

```
SELECT p.nombre AS producto, p.precio_venta, pr.titulo AS promocion, pr.descuento,
pr.estado
FROM t_producto p
INNER JOIN t_promocion pr ON p.id_promocion = pr.id_promocion;
```

-- 3. LEFT JOIN: Todos los productos, tengan o no promoción

```
SELECT p.nombre AS producto, p.precio_venta,
       IFNULL(pr.titulo, 'Sin promoción') AS promocion,
       IFNULL(pr.descuento, 0) AS descuento
FROM t_producto p
LEFT JOIN t_promocion pr ON p.id_promocion = pr.id_promocion;
```

-- 4. INNER JOIN: Usuarios con su ciudad

```
SELECT u.id_usuario, u.nombre, u.apellido, u.correo,
       c.ciudad, c.localidad, c.barrio, c.direccion AS direccion_ciudad
FROM t_usuario u
INNER JOIN t_ciudad c ON u.id_ciudad = c.id_ciudad;
```

-- 5. INNER JOIN: Facturas con cliente, vendedor y producto

```
SELECT f.id_factura, f.fecha,
       CONCAT(cli.nombre, ' ', cli.apellido) AS cliente,
       CONCAT(ven.nombre, ' ', ven.apellido) AS vendedor,
       p.nombre AS producto, f.cantidad, f.valor_unitario,
       (f.cantidad * f.valor_unitario) AS total, f.metodo_pago
FROM t_factura f
INNER JOIN t_usuario cli ON f.id_usuario_cliente = cli.id_usuario
INNER JOIN t_usuario ven ON f.id_usuario_vendedor = ven.id_usuario
INNER JOIN t_producto p ON f.id_producto = p.id_producto;
```

-- 6. INNER JOIN: Carrito con usuario y producto

```
SELECT ca.id_carrito, CONCAT(u.nombre,' ',u.apellido) AS usuario,  
       p.nombre AS producto, ca.cantidad, p.precio_venta,  
       (ca.cantidad * p.precio_venta) AS subtotal, ca.fecha_agregado  
FROM t_carrito ca  
INNER JOIN t_usuario u ON ca.id_usuario = u.id_usuario  
INNER JOIN t_producto p ON ca.id_producto = p.id_producto;
```

-- 7. INNER JOIN: Notificaciones con usuario y promoción

```
SELECT n.id_notificacion, CONCAT(u.nombre,' ',u.apellido) AS usuario,  
       u.correo, pr.titulo AS promocion, pr.descuento, n.fecha_envio  
FROM t_notificacion n  
INNER JOIN t_usuario u ON n.id_usuario = u.id_usuario  
INNER JOIN t_promocion pr ON n.id_promocion = pr.id_promocion;
```

-- 8. INNER JOIN: Reseñas con usuario y producto

```
SELECT r.id_resena, CONCAT(u.nombre,' ',u.apellido) AS usuario,  
       p.nombre AS producto, r.calificacion, r.comentario, r.fecha  
FROM t_resena r  
INNER JOIN t_usuario u ON r.id_usuario = u.id_usuario  
INNER JOIN t_producto p ON r.id_producto = p.id_producto;
```

-- 9. JOIN múltiple: Facturas completas con ciudad del cliente

```
SELECT f.id_factura, f.fecha, CONCAT(cli.nombre,' ',cli.apellido) AS cliente,  
       ciu.ciudad, ciu.localidad, p.nombre AS producto, cat.categoria,  
       f.cantidad, f.valor_unitario, (f.cantidad * f.valor_unitario) AS total  
FROM t_factura f  
INNER JOIN t_usuario cli ON f.id_usuario_cliente = cli.id_usuario  
INNER JOIN t_ciudad ciu ON cli.id_ciudad = ciu.id_ciudad  
INNER JOIN t_producto p ON f.id_producto = p.id_producto  
INNER JOIN t_categoria cat ON p.id_categoria = cat.id_categoria;
```

-- 10. JOIN + GROUP BY: Total vendido por vendedor

```
SELECT CONCAT(ven.nombre,' ',ven.apellido) AS vendedor,  
       COUNT(f.id_factura) AS total_ventas,  
       SUM(f.cantidad * f.valor_unitario) AS ingresos_totales  
FROM t_factura f  
INNER JOIN t_usuario ven ON f.id_usuario_vendedor = ven.id_usuario
```



```
GROUP BY ven.id_usuario, ven.nombre, ven.apellido;
```

```
-- 11. JOIN + GROUP BY: Calificación promedio por producto
```

```
SELECT p.nombre AS producto, cat.categoria,  
       ROUND(AVG(r.calificacion), 2) AS calificacion_promedio,  
       COUNT(r.id_resena) AS total_resenas  
FROM t_resena r  
INNER JOIN t_producto p ON r.id_producto = p.id_producto  
INNER JOIN t_categoria cat ON p.id_categoria = cat.id_categoria  
GROUP BY p.id_producto, p.nombre, cat.categoria;
```

```
-- 12. LEFT JOIN: Productos que nunca han sido vendidos
```

```
SELECT p.id_producto, p.nombre AS producto, p.precio_venta  
FROM t_producto p  
LEFT JOIN t_factura f ON p.id_producto = f.id_producto  
WHERE f.id_producto IS NULL;
```

6. Conclusiones

El desarrollo de este informe final permitió consolidar los conocimientos adquiridos a lo largo del proceso formativo en la asignatura de Bases de Datos, evidenciando la importancia de un diseño estructurado, coherente y normalizado para garantizar la integridad y eficiencia del almacenamiento de información.

A través del proceso de normalización hasta la Tercera Forma Normal (3FN), se logró eliminar redundancias y dependencias transitivas en las tablas, obteniendo un esquema limpio compuesto por nueve entidades: T_USUARIO, T_CIUDAD, T_CATEGORIA, T_PROMOCION, T_PRODUCTO, T_FACTURA, T_CARRITO, T_NOTIFICACION y T_RESENA. Cada tabla cumple con los principios según la normalización que se nos explicó en clases por la instructora presente que está leyendo dicho documento.

El Diagrama ER y el Modelo MER permitieron visualizar de manera gráfica las relaciones entre entidades, facilitando la comprensión de la estructura del sistema antes de su implementación. Estos modelos son herramientas fundamentales en el proceso de diseño, ya que permiten anticipar problemas de integridad referencial y definir con claridad las cardinalidades de cada relación. Cosa que cabe mencionar no se tenía tan presente ni se sabía bien su interpretación.

El script de base de datos, que incluye la creación de tablas con claves foráneas, restricciones de integridad y datos de prueba, demostró la implementación práctica del modelo diseñado. Las consultas SQL desarrolladas con diferentes tipos de JOIN (INNER JOIN, LEFT JOIN, JOIN múltiple y JOIN con GROUP BY) evidencian la capacidad del sistema para responder consultas complejas y generar reportes analíticos como ventas por vendedor y calificaciones promedio por producto.

En conclusión, el aprendizaje en el momento formativo nos llevó a entender más el concepto de bases de datos para poder fomentarlo e integrarlo a nuestro proyecto de mejor manera para su entrega final y poder hacer mejor nuestro proyecto para el cliente en su etapa final.